



Creación e Implementación de un Data Warehouse usando PostgreSQL y Jupyter con Python

Rosas Espinosa Alfredo

9B IDGS

Extracción de
Conocimiento en Bases
de Datos

MrySl. Maria Reina
Zarate Nava



GOBIERNO DEL ESTADO DE
VERACRUZ
2024 - 2030



POR AMOR A
VERACRUZ

SEV
SECRETARÍA
DE EDUCACIÓN
DE VERACRUZ

SEMSys
SUBSECRETARÍA DE EDUCACIÓN
MEDIA SUPERIOR Y SUPERIOR

DET
DIRECCIÓN
DE EDUCACIÓN
TECNOLÓGICA

SEP
SECRETARÍA
DE EDUCACIÓN
PÚBLICA

UTP
DIRECCIÓN GENERAL DE UNIVERSIDADES
TECNOLÓGICAS Y POLITÉCNICAS

Índice

Parte 1: Investigación Documental.....	3
1.1 ¿Qué es un Data Warehouse?	3
1.2 Diferencias: Base de Datos Operacional (OLTP) vs. Data Warehouse (OLAP).....	3
1.3 Ventajas del uso de un Data Warehouse	3
1.4 Arquitecturas Comunes de Modelado	4
1.5 Herramientas y tecnologías comunes	4
Parte 2: Instalación y Configuración (SQL)	4
2.1 Evidencia de la creación de la Base de Datos en pgAdmin	5
2.2 Evidencia del esquema de tablas creadas exitosamente	5
Parte 3: Carga de Datos y ETL (Python en Jupyter).....	6
3.1 Evidencia de configuración de librerías y cadena de conexión a la BD.....	6
3.2 Evidencia del output de Jupyter confirmando el éxito del pipeline ETL	7
Parte 4: Análisis, Consultas y Visualización	7
4.1 Evidencia del DataFrame resultante de la consulta OLAP estructurada.....	7
4.2 Evidencia del gráfico del Indicador Clave generado por Matplotlib	8
Parte 5: Conclusión	8

Parte 1: Investigación Documental

1.1 ¿Qué es un Data Warehouse?

Un Data Warehouse (Almacén de Datos) es un sistema de almacenamiento electrónico que recolecta, consolida y organiza datos estructurados de múltiples fuentes de una organización. Su propósito exclusivo es el análisis de datos, la generación de reportes y el soporte para la toma de decisiones estratégicas, aislando estas consultas del entorno operativo diario.

1.2 Diferencias: Base de Datos Operacional (OLTP) vs. Data Warehouse (OLAP)

Característica	Base de Datos Operacional (OLTP)	Data Warehouse (OLAP)
Propósito	Registrar transacciones del día a día (Altas, Bajas, Cambios).	Análisis de tendencias y soporte de decisiones.
Diseño	Altamente normalizado (Evita redundancia).	Desnormalizado (Optimizado para lecturas rápidas).
Operaciones	Escrituras y actualizaciones constantes y rápidas.	Lecturas masivas de datos históricos (Inserciones periódicas).
Datos	Datos actuales, detallados y dinámicos.	Datos históricos, consolidados y estáticos.

1.3 Ventajas del uso de un Data Warehouse

- **Consolidación e Integración:** Unifica datos provenientes de diferentes plataformas o formatos en un solo lugar.
- **Rendimiento de Consultas:** Optimiza la velocidad de respuesta en consultas e indicadores analíticos complejos sin afectar el sistema de producción.
- **Análisis Histórico:** Permite realizar análisis de tendencias comparando datos a lo largo de meses o años.

1.4 Arquitecturas Comunes de Modelado

- **Modelo Estrella (Star Schema):** Consta de una tabla de hechos central que contiene las métricas cuantitativas, conectada directamente a múltiples tablas de dimensiones mediante claves foráneas. Es el más simple y eficiente para consultas rápidas.
- **Modelo Copo de Nieve (Snowflake Schema):** Es una variante del modelo estrella donde algunas tablas de dimensiones se normalizan y se dividen en subtablas, asemejando la forma de un copo de nieve. Ahorra espacio pero ralentiza las consultas debido a los JOINS adicionales.
- **Constelación de Hechos (Fact Constellation):** Estructura avanzada que contiene múltiples tablas de hechos que comparten una o más tablas de dimensiones comunes, ideal para modelar flujos empresariales complejos.

1.5 Herramientas y tecnologías comunes

- **Bases de Datos (Motores):** PostgreSQL, Amazon Redshift, Google BigQuery, Snowflake.
- **Herramientas ETL / Lenguajes:** Python (Pandas, SQLAlchemy), Apache Hop, Talend.
- **Visualización (BI):** Power BI, Tableau, Looker Studio.

Parte 2: Instalación y Configuración (SQL)

En esta sección se detalla el proceso de inicialización del entorno en PostgreSQL. Primero se realiza la conexión al servidor local y la creación de una base de datos dedicada exclusivamente para el almacenamiento analítico. Posteriormente, se ejecuta un script estructurado bajo el enfoque de Modelo **Estrella (Star Schema)**, el cual inicializa de forma íntegra las tres tablas de dimensiones principales (***dim_cliente***, ***dim_producto*** y ***dim_tiempo***) y la tabla de hechos central (***hechos_ventas***) encargada de consolidar las métricas de la organización mediante claves foráneas.

2.1 Evidencia de la creación de la Base de Datos en pgAdmin

```
1 CREATE DATABASE data_warehouse;
```

Data Output Messages Notifications

CREATE DATABASE

Query returned successfully in 252 msec.

2.2 Evidencia del esquema de tablas creadas exitosamente

```
1 -- 1. Creación de Tablas de Dimensiones
2 CREATE TABLE dim_cliente (
3     cliente_id SERIAL PRIMARY KEY,
4     nombre VARCHAR(100),
5     ciudad VARCHAR(50),
6     categoria VARCHAR(30)
7 );
8
9 CREATE TABLE dim_producto (
10     producto_id SERIAL PRIMARY KEY,
11     nombre_producto VARCHAR(100),
12     categoria_prod VARCHAR(50),
13     precio_unitario NUMERIC(10, 2)
14 );
15
16 CREATE TABLE dim_tiempo (
17     tiempo_id SERIAL PRIMARY KEY,
18     fecha DATE UNIQUE,
19     anio INT,
20     mes INT,
21     dia INT
22 );
23
24 -- 2. Creación de la Tabla de Hechos (Modelo Estrella)
25 CREATE TABLE hechos_ventas (
26     venta_id SERIAL PRIMARY KEY,
27     cliente_id INT REFERENCES dim_cliente(cliente_id),
28     producto_id INT REFERENCES dim_producto(producto_id),
29     tiempo_id INT REFERENCES dim_tiempo(tiempo_id),
30     cantidad INT,
31     monto_total NUMERIC(12, 2)
32 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 166 msec.

Parte 3: Carga de Datos y ETL (Python en Jupyter)

En esta sección se detalla la implementación del proceso ETL (Extracción, Transformación y Carga) automatizado en Python dentro del entorno de Jupyter Notebook. Utilizando la librería Pandas, se estructuran los conjuntos de datos de prueba para las dimensiones, aplicando una lógica de normalización temporal sobre la dimensión de tiempo para desglosar fechas automáticas por año, mes y día. Finalmente, la integración con SQLAlchemy permite establecer una conexión directa y segura hacia el servidor de PostgreSQL para realizar la inyección masiva de los registros dentro de las tablas correspondientes.

3.1 Evidencia de configuración de librerías y cadena de conexión a la BD

```
[5]: #Instalación de la herramienta para SQL
#pip install sqlalchemy
#pip install psycopg2-binary

[3]: #Importamos librerías
import pandas as pd
import numpy as np
from sqlalchemy import create_engine

[4]: # Conexión segura a PostgreSQL
engine = create_engine('postgresql://postgres:1234567@localhost:5432/data_warehouse')

[7]: # Generación de datos simulados y carga de dimensiones
data_clientes = {
    'nombre': ['Ana López', 'Luis Pérez', 'Carlos Gómez', 'Sofía Ruiz'],
    'ciudad': ['Córdoba', 'Fortín', 'Orizaba', 'Veracruz'],
    'categoria': ['Premium', 'Estándar', 'Premium', 'Estándar']
}
df_clientes = pd.DataFrame(data_clientes)
df_clientes.to_sql('dim_cliente', engine, if_exists='append', index=False)

data_productos = {
    'nombre_producto': ['Laptop Pro', 'Teclado Mecánico', 'Monitor 4K', 'Mouse Gamer'],
    'categoria_prod': ['Cómputo', 'Accesorios', 'Video', 'Accesorios'],
    'precio_unitario': [18500.00, 1200.00, 7500.00, 650.00]
}
df_productos = pd.DataFrame(data_productos)
df_productos.to_sql('dim_producto', engine, if_exists='append', index=False)

[7]: 4

[8]: # Transformación y Normalización de la Dimensión Tiempo
fechas = pd.date_range(start='2026-01-01', end='2026-03-31')
df_tiempo = pd.DataFrame({
    'fecha': fechas,
    'anio': fechas.year,
    'mes': fechas.month,
    'dia': fechas.day
})
df_tiempo.to_sql('dim_tiempo', engine, if_exists='append', index=False)

[8]: 90

[9]: # Simulación y carga de la Tabla de Hechos
np.random.seed(42)
registros_hechos = {
    'cliente_id': np.random.choice([1, 2, 3, 4], size=100),
    'producto_id': np.random.choice([1, 2, 3, 4], size=100),
    'tiempo_id': np.random.choice(range(1, len(fechas) + 1), size=100),
    'cantidad': np.random.randint(1, 5, size=100)
}
df_hechos = pd.DataFrame(registros_hechos)

[10]: # Mapeo para calcular el monto_total basado en el precio unitario
precios = {1: 18500.00, 2: 1200.00, 3: 7500.00, 4: 650.00}
df_hechos['monto_total'] = df_hechos['producto_id'].map(precios) * df_hechos['cantidad']
```

3.2 Evidencia del output de Jupyter confirmando el éxito del pipeline ETL

```
[11]: # Carga de tabla de hechos final
df_hechos.to_sql('hechos_ventas', engine, if_exists='append', index=False)
print("✅ Carga masiva al Data Warehouse finalizada con éxito.")

✅ Carga masiva al Data Warehouse finalizada con éxito.
```

Parte 4: Análisis, Consultas y Visualización

En esta última etapa, se demuestra la utilidad del Data Warehouse mediante la ejecución de consultas analíticas complejas utilizando operaciones de agregación y uniones estructurales (**JOINS**). Los datos consolidados se extraen directamente de PostgreSQL hacia Python para interpretar los indicadores clave de rendimiento (KPIs). Finalmente, se utiliza la librería **Matplotlib** para procesar un gráfico de barras que permite visualizar y comparar de forma clara y legible el comportamiento de las métricas comerciales distribuidas por regiones geográficas.

4.1 Evidencia del DataFrame resultante de la consulta OLAP estructurada

```
[12]: # Consulta SQL Analítica - Total de ventas por Ciudad
query_ciudades = """
SELECT c.ciudad, SUM(h.monto_total) AS total_ventas
FROM hechos_ventas h
JOIN dim_cliente c ON h.cliente_id = c.cliente_id
GROUP BY c.ciudad
ORDER BY total_ventas DESC;
"""
df_analisis_ciudad = pd.read_sql(query_ciudades, engine)
print(df_analisis_ciudad)
```

	ciudad	total_ventas
0	Orizaba	744400.0
1	Veracruz	554950.0
2	Fortín	424550.0
3	Córdoba	342800.0

4.2 Evidencia del gráfico del Indicador Clave generado por Matplotlib



Parte 5: Conclusión

La realización de este proyecto permitió comprender de manera práctica y metodológica la importancia de la transición entre los sistemas transaccionales tradicionales (OLTP) y los entornos analíticos desnormalizados (OLAP). A través del diseño e implementación de un **Data Warehouse** bajo la arquitectura de **Modelo Estrella**, se constató cómo la separación de las operaciones analíticas optimiza drásticamente los tiempos de respuesta en consultas complejas, evitando la sobrecarga en los servidores de producción de una organización. Asimismo, la integración de herramientas como **PostgreSQL** para el almacenamiento robusto, junto con **Python (Pandas y SQLAlchemy)** en **Jupyter** para la automatización del pipeline ETL, demostró que la calidad de las decisiones estratégicas de una empresa depende directamente de la correcta extracción, depuración y transformación de sus datos. En conclusión, el desarrollo de un Almacén de Datos eficiente no solo centraliza la información histórica, sino que la transforma en un activo estratégico legible, interactivo y de alto valor para la gobernanza de datos y la inteligencia de negocios (BI).